

AiTakeaway AI 点餐助手系统设计报告

1. 引言

1.1 编写目的

本文档在立项报告和需求分析报告的基础上，对 AiTakeaway 系统的设计进行说明，内容包括总体架构、模块划分、关键设计原则、数据库结构、接口与权限设计以及核心业务流程，作为后续实现、测试和维护的依据。文档按“概要设计—详细设计”两级展开：概要设计说明系统整体如何组织，详细设计说明关键模块内部如何工作、对象如何协作、数据如何流动。

1.2 项目概述

AiTakeaway 是一个基于 Spring Boot 的 AI 点餐助手系统，面向普通用户和商家两类角色，提供注册登录、店铺与菜品管理、购物车、下单和订单状态流转等点餐能力。在此之上，系统接入了基于 LangChain4j 与 DeepSeek（OpenAI 兼容接口）的 AI 助手，让用户能用自然语言查询菜品、表达点餐意图并辅助完成下单。

后端使用 Spring Boot 3.3.5、Java 17、Spring Security、JWT、MyBatis-Plus、MySQL、Redis 和 LangChain4j，代码按 controller、service、mapper、entity、dto、config、common 等包组织，是一个典型的分层 Web 应用。

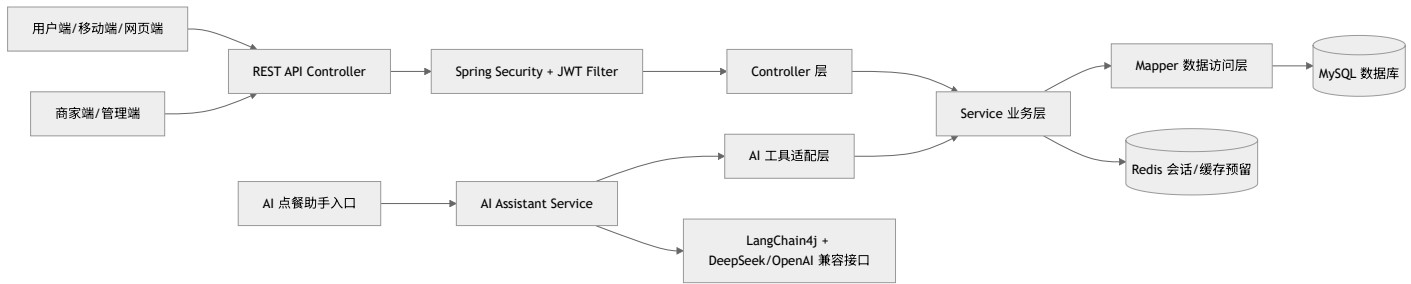
1.3 设计目标

设计围绕以下几点展开：点餐主流程（浏览、加购、下单、商家确认配送）清晰可靠；用户端与商家端接口隔离，敏感操作必须经过 JWT 认证和角色授权；各模块职责清晰、通过接口协作，保持高内聚低耦合；下单、结算、状态变更等关键操作事务保证数据一致；AI 助手通过业务服务或工具适配层访问点餐能力，而非直接读写数据库，避免与底层结构强耦合；同时通过统一响应、统一异常处理和统一数据访问方式降低重复代码，提升可维护性。

2. 概要设计

2.1 系统总体架构

系统采用前后端分离下的后端分层架构。客户端通过 HTTP/JSON 调用 REST API，请求经 Spring MVC 进入 Controller，由 Service 处理业务逻辑，再经 MyBatis-Plus Mapper 访问 MySQL。安全认证由 Spring Security 和 JWT 过滤器完成；AI 能力通过 LangChain4j 对接大语言模型，并以独立适配层的方式调用业务服务。



2.2 分层职责设计

层次	主要包/组件	设计职责
表现层	controller	接收请求、解析参数、获取当前用户身份、调用 Service、返回统一 Result
业务层	service、service.impl	业务规则、权限归属校验、状态流转、事务控制、跨模块协作
数据访问层	mapper	基于 MyBatis-Plus BaseMapper 提供 CRUD
实体层	entity	对应数据库表，含状态常量、逻辑删除、自动填充字段
传输对象层	dto	封装登录、注册、订单详情等输入输出结构
公共支撑层	common	统一响应、JWT 工具、全局异常处理
配置层	config	Spring Security、JWT 过滤器、MyBatis-Plus 配置
AI 能力层	LangChain4j 配置与 AI 服务	对接大模型，将自然语言点餐意图转为可控的业务调用

这样的分层让 Controller 不直接碰 Mapper、AI 助手不直接碰数据库，业务规则集中在 Service 层。

2.3 模块划分

系统按业务领域分为六个模块。

模块	主要类	核心职责
认证	AuthController、UserServiceImpl、JwtUtil、JwtAuthenticationFilter	注册、登录、密码加密、JWT 生成与校验
商家	MerchantController、MerchantServiceImpl、Merchant	店铺创建修改、营业状态、查询与搜索
菜品	DishController、DishServiceImpl、Dish	菜品增删改、上下架、商家与用户视角的列表
购物车	CartController、CartServiceImpl、Cart	加购、改数量、删除、清空、结算成订单
订单	OrderController、OrderServiceImpl、Order、OrderItem	下单、订单详情、用户/商家订单列表、状态流转
AI 助手	LangChain4j、DeepSeek 配置、AI 服务适配	理解自然语言点餐意图，查询店铺/菜品，辅助构建点餐方案

2.4 内聚与耦合分析

各模块以功能内聚和信息内聚为主：认证模块围绕用户身份认证，商家、菜品模块分别围绕 Merchant、Dish 聚合资料维护与归属校验，购物车模块围绕"临时选餐"管理购物车项，订单模块按校验—计算—保存订单头—保存明细的顺序组织，AI 助手模块则围绕一次对话上下文把自然语言、用户状态、候选菜品和工具调用串联起来。

模块之间以数据耦合和接口耦合为主，通过 Service 接口传递参数或领域对象，不触碰彼此的内部实现。

协作关系	耦合方式	说明
Controller → Service	接口耦合	Controller 只依赖服务接口，便于替换实现
Service → Mapper	数据访问耦合	CRUD 统一交给 MyBatis-Plus，避免 SQL 散落在业务层
CartService → OrderService	接口耦合	结算时复用下单逻辑，不重复实现
OrderService → DishService/MerchantService	接口耦合	下单时复用菜品、商家查询，保证规则一致
AI 助手 → 业务服务	适配层耦合	AI 不直接访问数据库，降低模型输出不确定性对核心业务的影响

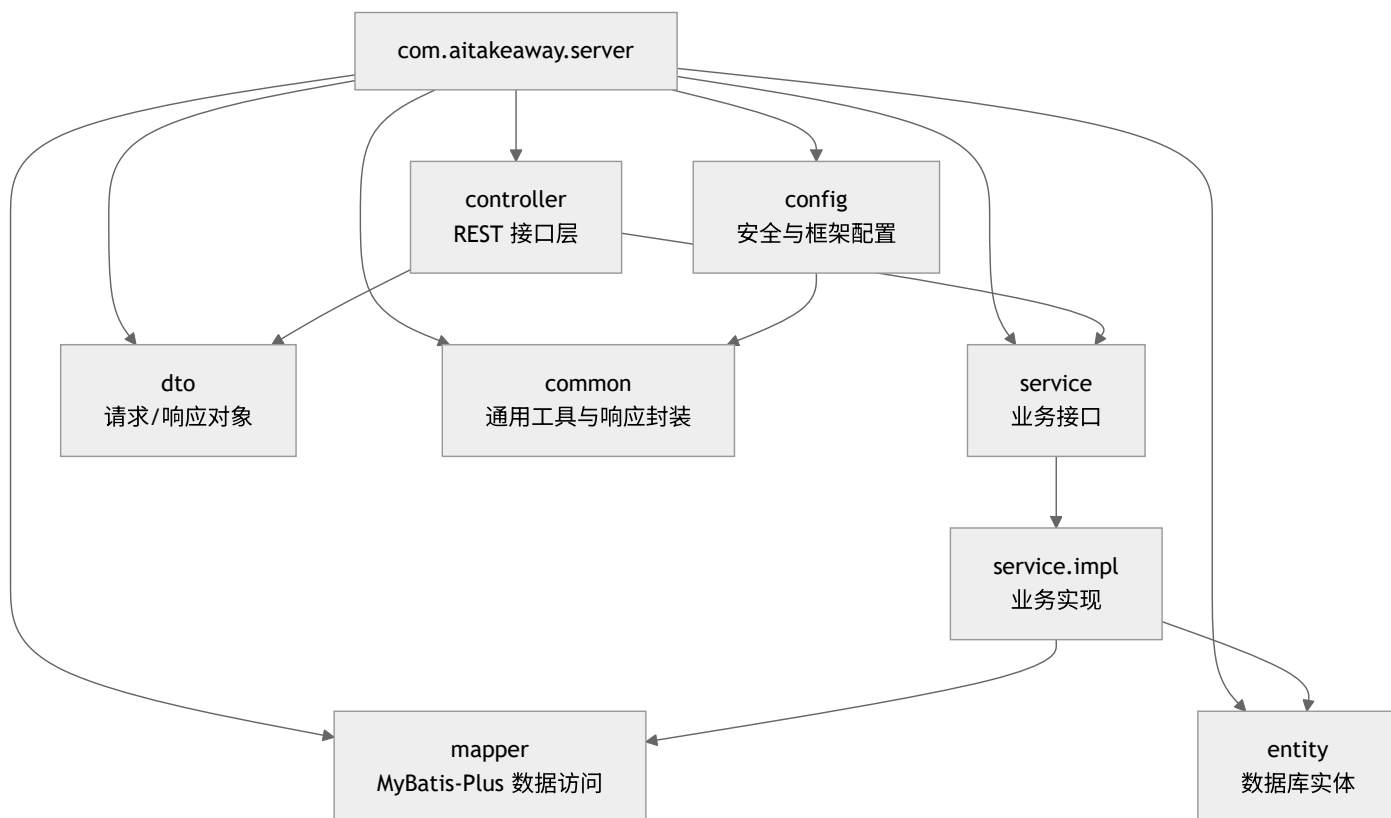
耦合控制的关键在于：业务模块之间不共享数据库访问细节，跨模块操作一律走公开服务方法；订单保留明细快照，避免历史订单受菜品后续修改影响。

2.5 设计原则应用

- **单一职责**：每个 Controller 只暴露一个资源域的 API，业务规则归 Service，数据访问归 Mapper，认证过滤归 JwtAuthenticationFilter。
- **开闭与依赖倒置**：Controller 依赖 UserService、OrderService 等接口而非实现；新增如优惠券能力时，只需增加 CouponService 并在订单计算处接入，AI 助手也可通过新增工具适配器扩展可调用能力，无需改动整体结构。
- **接口隔离**：服务接口按模块拆分，避免出现"全能服务"——用户服务只管登录注册，订单服务只管下单和状态流转。
- **信息隐藏**：JWT 密钥、数据库与模型配置都放在配置文件，由工具类或框架读取；Controller 不关心 Token 签名，Service 不关心 HTTP 细节，Mapper 不关心业务流程。
- **防御式设计**与**事务一致性**：下单、结算、状态变更等操作加 @Transactional，并在业务方法中对商家状态、菜品状态、数量、订单归属与状态做前置校验，拦住非法数据。

3. 系统架构设计

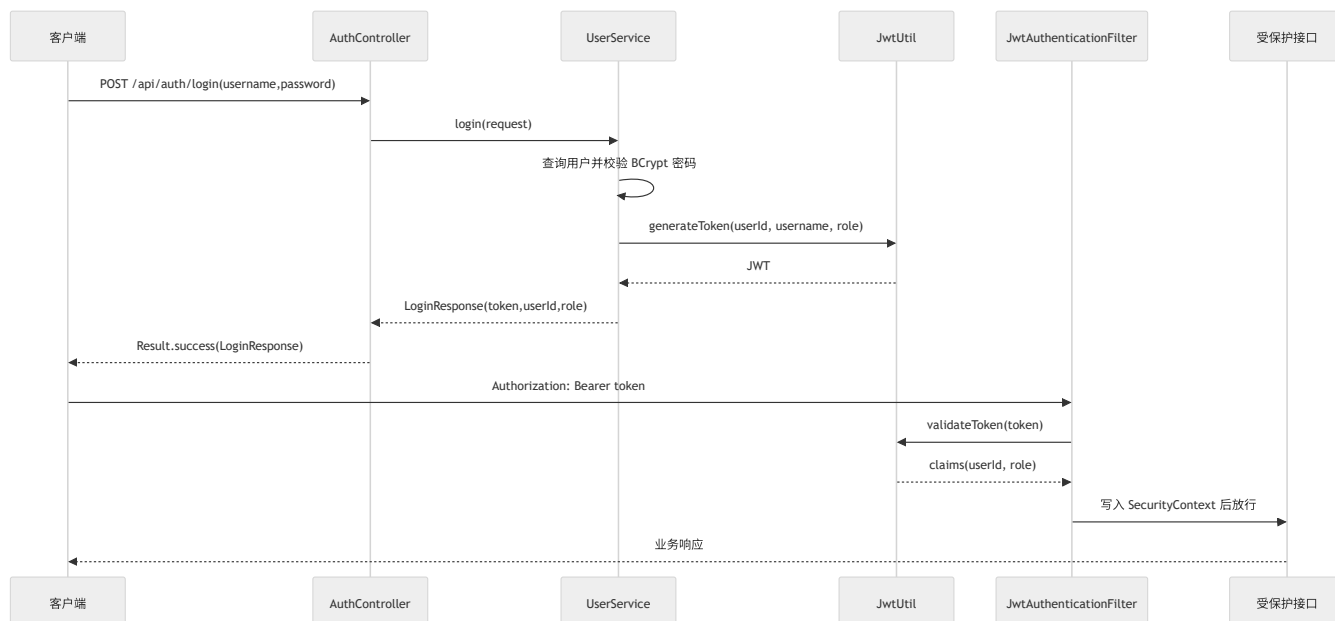
3.1 包结构



3.2 权限架构

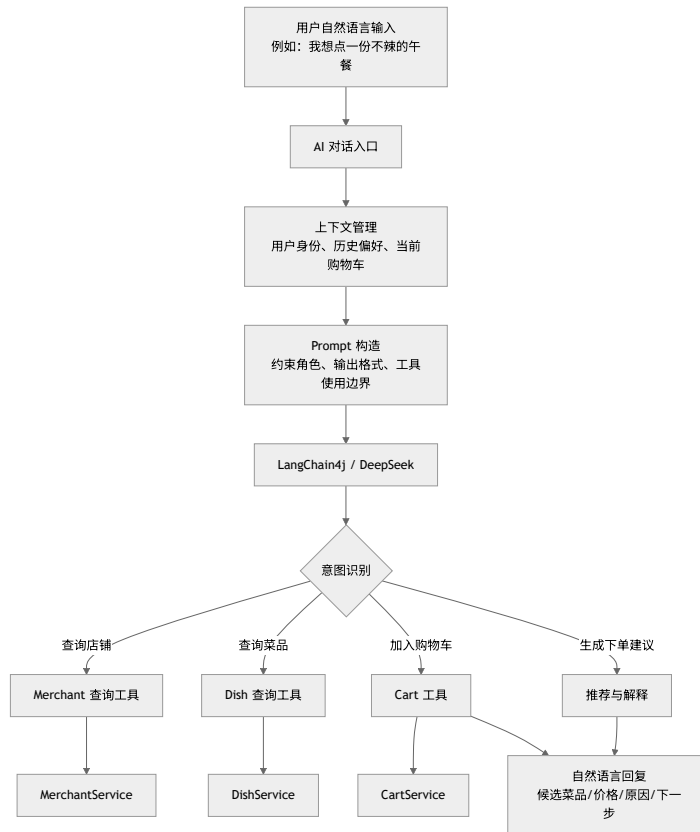
系统采用无状态 JWT 认证。用户登录成功后服务端签发 JWT，客户端在后续请求的 `Authorization: Bearer <token>` 头中携带；`JwtAuthenticationFilter` 解析 Token，把 `userId` 作为 principal 写入 `SecurityContextHolder`，并按 Token 中的 `role` 构造 `ROLE_CUSTOMER`、`ROLE_MERCHANT` 等权限。

权限控制分两层。接口层做粗粒度控制，由 `SecurityConfig` 按 URL 和 HTTP 方法限制角色，例如建店、菜品维护、确认订单等接口要求 `MERCHANT` 角色；业务层做细粒度控制，`Service` 校验数据归属，商家只能改自己的店铺菜品、处理自己店铺的订单，用户只能查看和取消自己的订单。



3.3 AI 助手架构

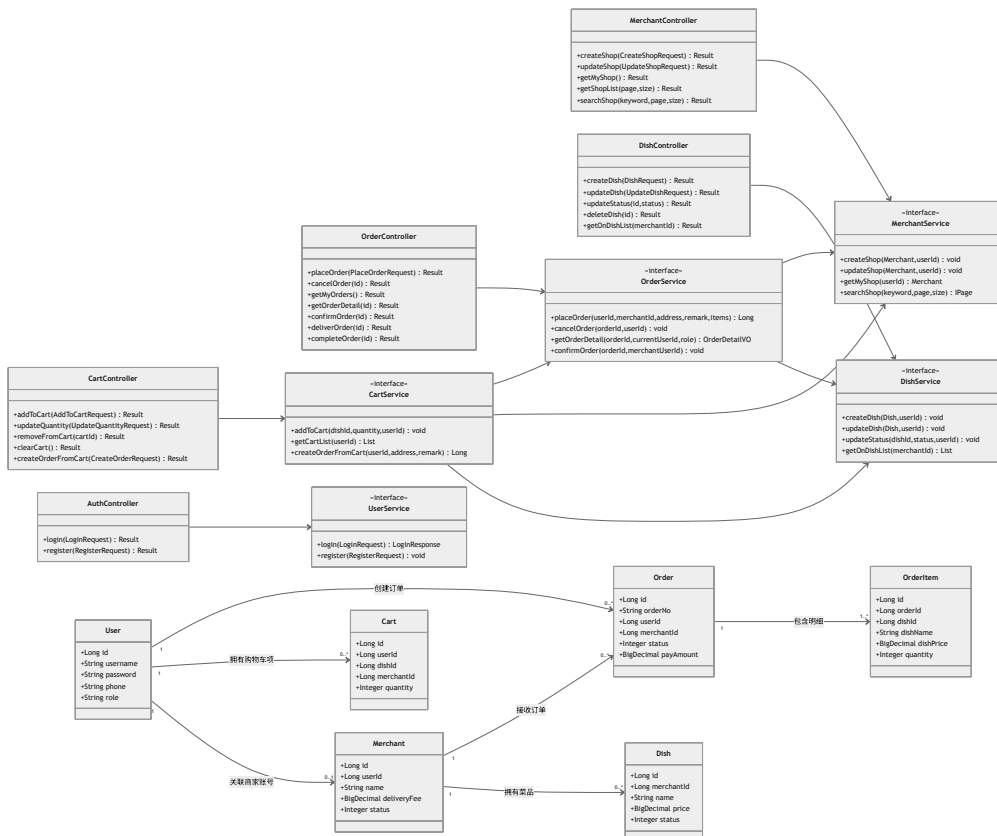
AI 助手的设计要点是“让 AI 参与点餐决策，但不绕过业务规则”——它不直接写数据库、不拼 SQL、不直接改订单状态，只通过受控的业务服务或工具适配器完成查询和操作。



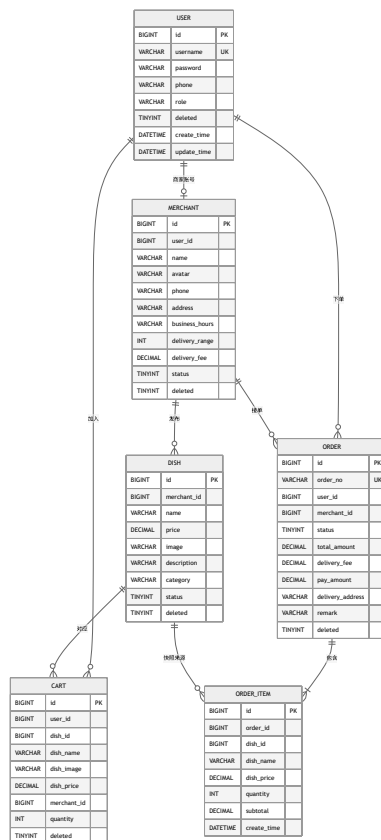
具体约束有几条：AI 只做理解、推荐和辅助构建，不绕过权限系统；它发起的购物车、订单动作必须绑定当前登录用户；推荐基于店铺和菜品服务的实时数据，不会推已下架菜品或休息中的商家；最终下单、取消等关键操作必须由用户确认；调用外部模型失败时不影响普通点餐主流程。

4. 详细设计

4.1 设计类图



4.2 数据库 E-R 设计

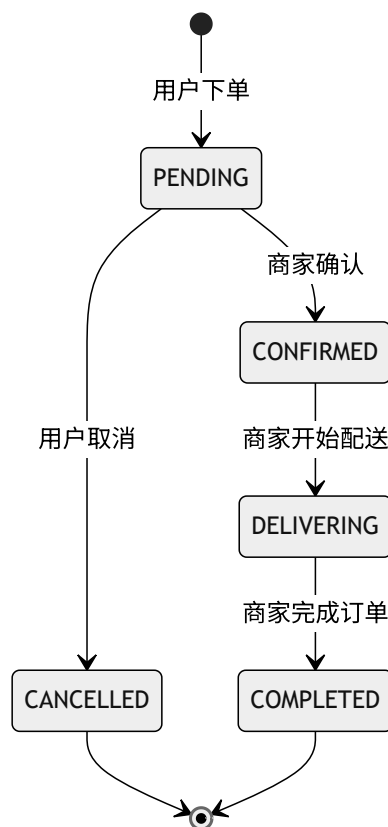


设计上有几点值得说明：`user.username` 唯一以避免重复账号，`merchant.user_id` 把店铺与商家账号一对一关联，`dish.merchant_id` 标明菜品归属。`cart` 冗余了菜品名称、图片和价格，`order_item` 则保存了名称和单价的快照——前者提升购物车展示效率，后者保证菜品后续改名或调价不会影响历史订单。多个业务表带 `deleted` 字段配合 MyBatis-Plus 逻辑删除，`create_time` / `update_time` 由 MyBatis-Plus 自动填充。

4.3 核心状态设计

菜品有下架（`STATUS_OFF`，0）和上架（`STATUS_ON`，1）两种状态，只有上架菜品用户才能加购下单；商家有休息中（`STATUS_REST`，0）和营业中（`STATUS_OPEN`，1），休息时不可下单。

订单状态按有限状态机流转，Service 层通过 `checkStatus` 校验当前状态，防止越权或越序变更。

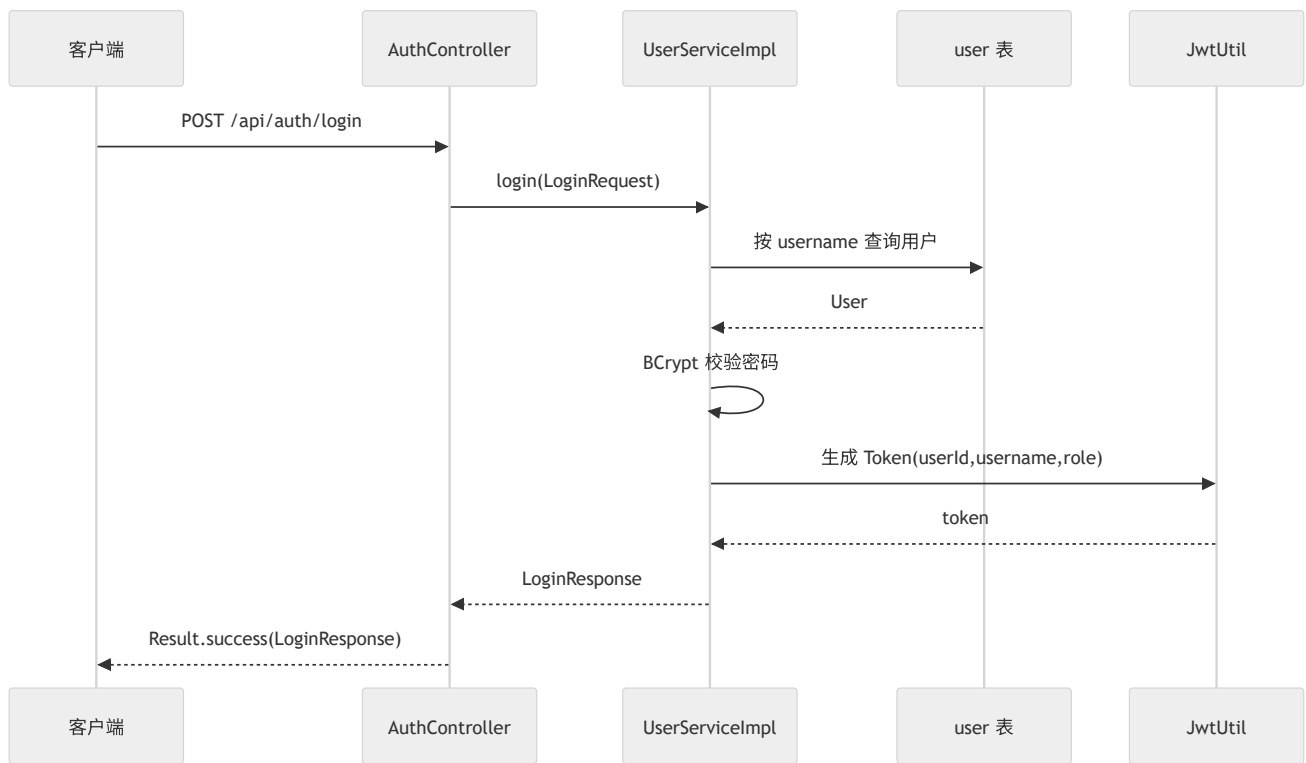


状态值	常量	含义	触发者
0	STATUS_PENDING	待确认	用户下单后自动进入
1	STATUS_CONFIRMED	已确认	商家确认订单
2	STATUS_DELIVERING	配送中	商家开始配送
3	STATUS_COMPLETED	已完成	商家确认完成
4	STATUS_CANCELLED	已取消	用户在待确认阶段取消

5. 核心业务详细设计

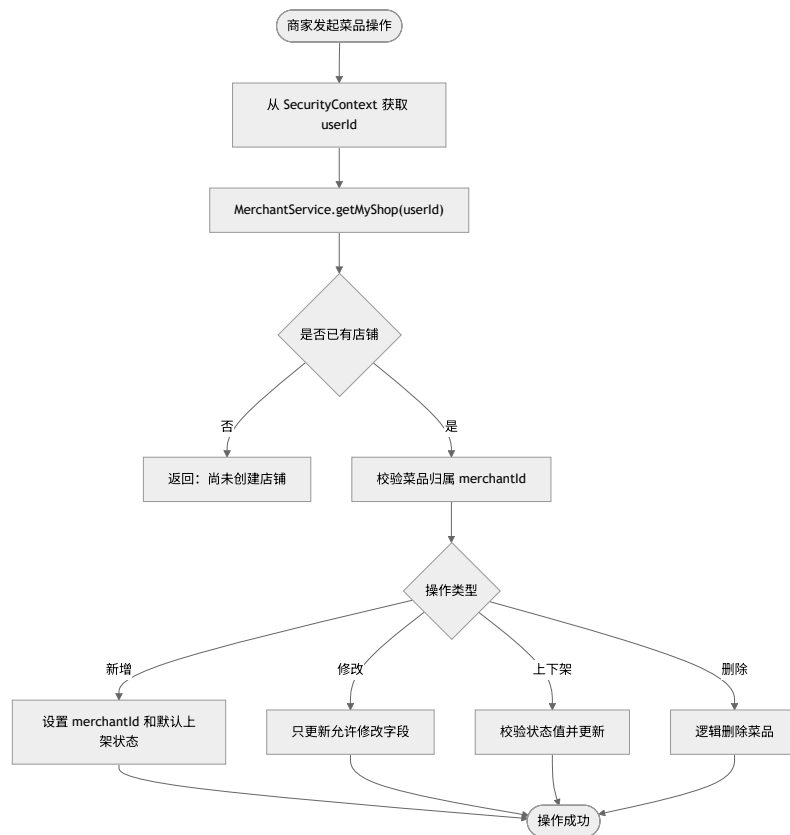
5.1 注册与登录

注册时先检查用户名是否已存在，不存在则用 BCrypt 加密密码后保存。登录时按用户名查出用户并校验密码，通过后生成 JWT 返回客户端。



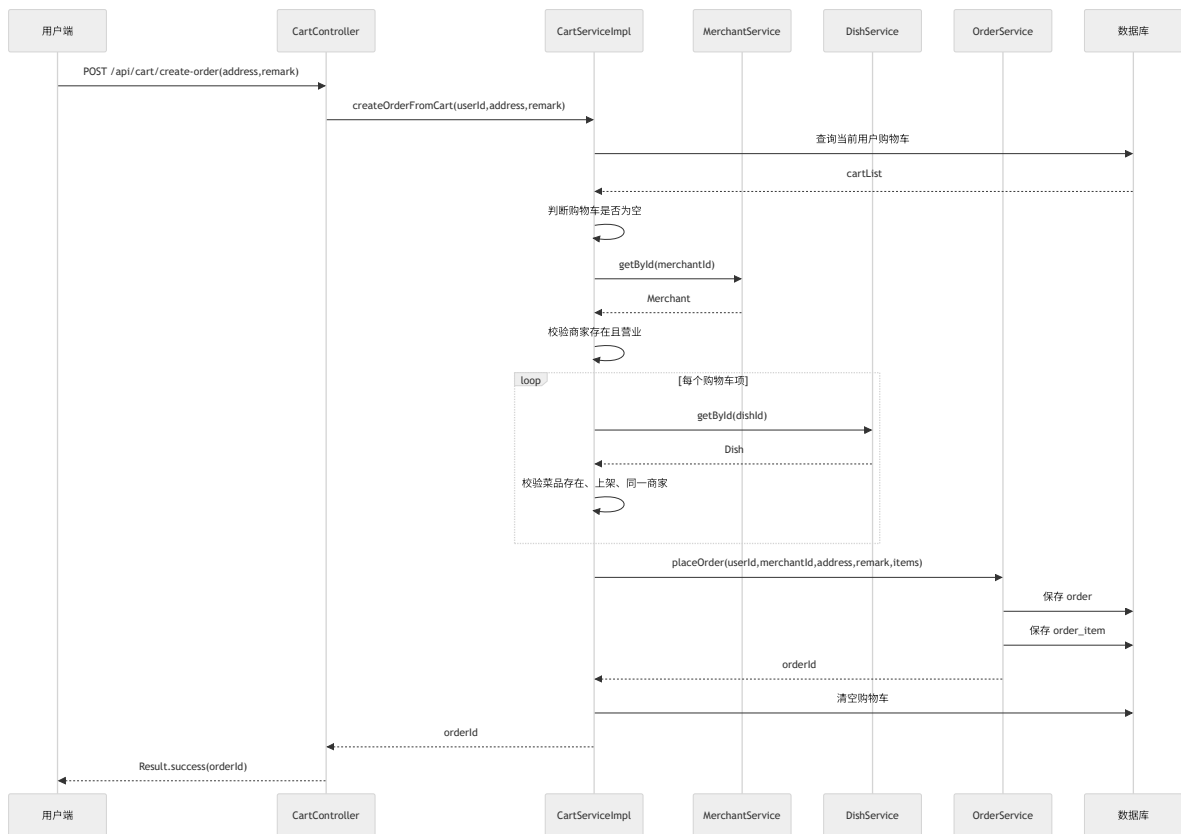
5.2 商家与菜品管理

菜品操作前必须先用当前商家账号找到所属店铺，再校验菜品是否属于该店铺。归属校验全部集中在 `DishServiceImpl`，Controller 不需要知道判断细节。



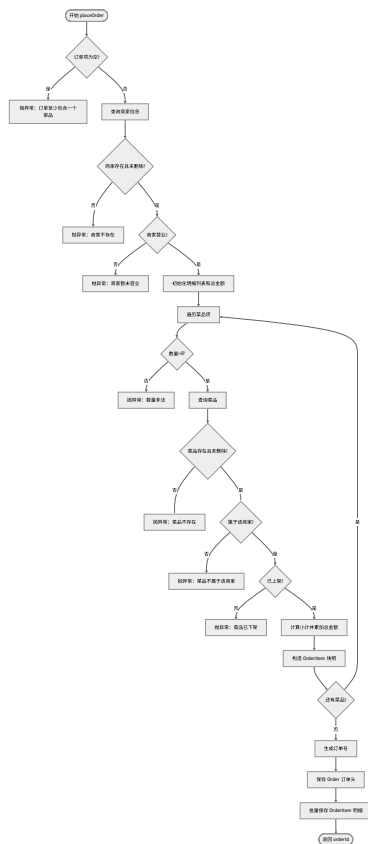
5.3 购物车结算成订单

从购物车下单要跨越购物车、商家、菜品、订单四个模块，是系统里比较典型的模块协作流程：查出购物车后校验非空，再校验商家存在且营业，逐项校验菜品有效，最后复用 `OrderService.placeOrder` 创建订单并清空购物车。



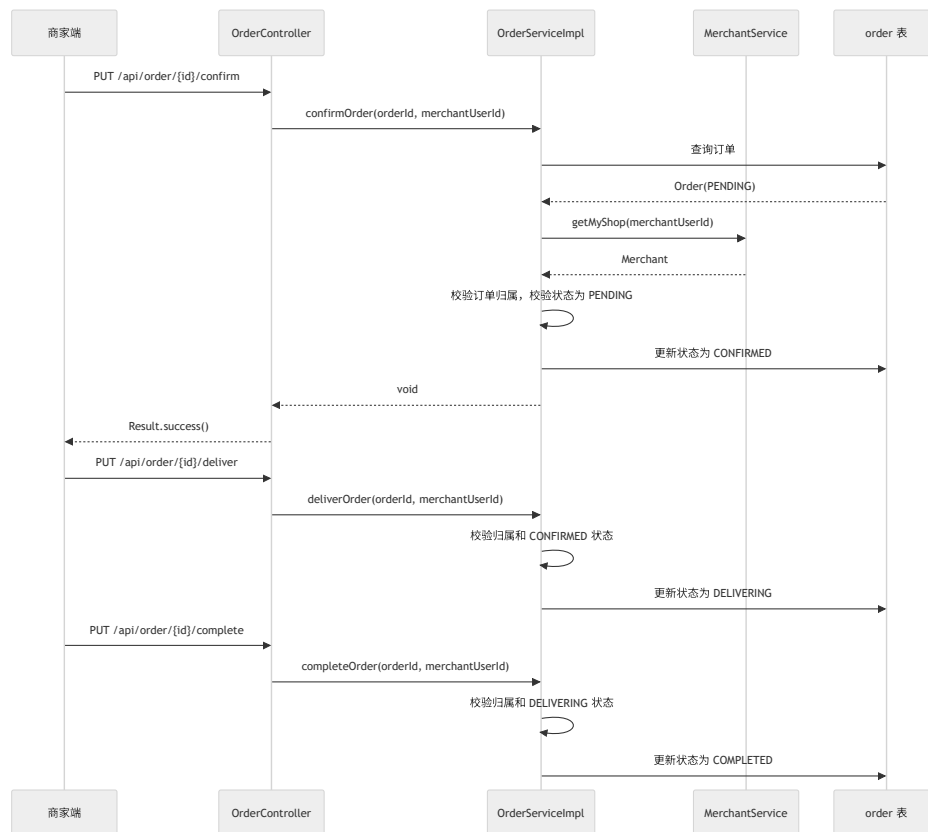
5.4 下单流程

下单是系统的关键流程：要同时校验商家、菜品、数量和价格，并在同一事务中保存订单头与明细。



5.5 订单状态流转

商家依次确认、配送、完成订单，每一步都先校验订单归属和当前状态，再更新到下一个状态。



6. 接口设计

6.1 统一响应格式

接口响应统一用 `Result<T>` 封装：`code`（成功 200，异常默认 500）、`message`（响应消息）、`data`（响应数据）。

```
{
  "code": 200,
  "message": "success",
  "data": { "orderId": 10001 }
}
```

6.2 主要 REST 接口

认证

方法	路径	权限	功能
POST	<code>/api/auth/register</code>	公开	用户注册
POST	<code>/api/auth/login</code>	公开	登录并返回 JWT

商家

方法	路径	权限	功能
GET	<code>/api/merchant/my-shop</code>	MERCHANT	查询我的店铺
POST	<code>/api/merchant/create</code>	MERCHANT	创建店铺
PUT	<code>/api/merchant/update</code>	MERCHANT	更新店铺
PUT	<code>/api/merchant/status</code>	MERCHANT	修改营业状态
GET	<code>/api/merchant/detail/{id}</code>	登录用户	查询店铺详情
GET	<code>/api/merchant/list</code>	登录用户	分页查询营业店铺
GET	<code>/api/merchant/search</code>	登录用户	按关键字搜索店铺

菜品

方法	路径	权限	功能
POST	/api/dish/create	MERCHANT	创建菜品
PUT	/api/dish/update	MERCHANT	更新菜品
PUT	/api/dish/status	MERCHANT	上下架菜品
DELETE	/api/dish/{id}	MERCHANT	删除菜品
GET	/api/dish/my-list	MERCHANT	查询自己店铺菜品
GET	/api/dish/list/{merchantId}	登录用户	查询指定店铺上架菜品

购物车

方法	路径	权限	功能
POST	/api/cart/add	登录用户	添加菜品到购物车
PUT	/api/cart/update-quantity	登录用户	修改购物车数量
DELETE	/api/cart/{cartId}	登录用户	删除购物车项
DELETE	/api/cart/clear	登录用户	清空购物车
GET	/api/cart/list	登录用户	查询购物车列表
POST	/api/cart/create-order	登录用户	从购物车创建订单

订单

方法	路径	权限	功能
POST	/api/order/place	CUSTOMER	用户直接下单
PUT	/api/order/{id}/cancel	CUSTOMER	取消待确认订单
GET	/api/order/my-list	CUSTOMER	查询自己的订单
GET	/api/order/{id}	登录用户	查询订单详情（Service 层再判归属）
GET	/api/order/merchant-list	MERCHANT	商家查询店铺订单
PUT	/api/order/{id}/confirm	MERCHANT	确认订单
PUT	/api/order/{id}/deliver	MERCHANT	开始配送
PUT	/api/order/{id}/complete	MERCHANT	完成订单

7. 异常处理与事务设计

7.1 异常处理

GlobalExceptionHandler 统一捕获业务运行时异常并返回 Result.error(message)。业务层用抛 RuntimeException 表示规则失败，常见的有：用户名或密码错误；无权操作他人的店铺、菜品、购物车或订单；商家不存在或未营业；菜品不存在、已下架或不属于该商家；订单状态不满足当前操作。统一处理后 Controller 不必到处写 try-catch，返回格式也保持一致。

7.2 事务设计

建店、改店、菜品增删改、购物车增删改、购物车结算下单、用户直接下单、订单状态变更等操作都加了事务。其中下单最关键——订单头保存成功而明细失败会留下脏数据，因此两者必须在同一事务里同时成功或同时回滚。

8. 数据一致性与安全设计

数据一致性靠几条策略保证：下单时保存菜品名称和单价快照，让历史订单不受后续修改影响；购物车冗余菜品信息以减少展示时的跨表查询；下单时重新校验菜品实时状态，避免用过期购物车买到已下架菜品；状态流转受有限状态机约束，不能从"待确认"直接跳到"已完成"；逻辑删除用于保护历史数据。

安全方面：密码用 BCrypt 加密存储，不存明文；JWT 只放用户 ID、用户名、角色等必要信息；Spring Security 隔离商家与用户接口，Service 层再做数据归属校验，防止伪造 URL 操作他人数据；AI 助手只通过工具适配层调用业务服务，不直接写库；数据库密码、模型 API Key 等敏感配置在正式部署时应迁移到环境变量或密钥管理，不提交到公开仓库。

9. 可维护性与可扩展性

开发上约定 Controller 只做参数接收、身份获取和服务调用，Service 负责业务规则、事务和跨模块协作，Mapper 只做数据访问，Entity 与表字段一致且状态用常量表达，DTO 隔离输入输出、避免直接暴露敏感实体字段。

系统也为后续能力预留了接入点：

扩展能力	接入方式
AI 菜品推荐	经 AI 工具适配层调用 MerchantService / DishService 取候选数据
AI 自动加购	AI 给建议，用户确认后调用 CartService.addToCart
优惠券/满减	在订单金额计算处引入 PromotionService
支付模块	订单确认后增加支付状态和支付流水表
评价模块	新增 Review 实体与 ReviewService，关联用户、商家、订单
配送员模块	在订单状态机中扩展接单、取餐、送达等状态

10. 测试设计建议

单元测试重点覆盖 Service 层规则：注册用户名重复、登录密码错误、商家重复建店、菜品归属校验、加购时菜品下架、下单时的空订单/跨商家菜品/下架菜品/非法数量、订单状态流转合法性。

接口测试覆盖角色权限边界：

场景	期望
未登录访问购物车接口	401
CUSTOMER 访问商家菜品维护接口	403
MERCHANT 访问用户下单接口	403
用户查看自己的订单	成功
用户查看他人订单	业务层拒绝
商家处理非自己店铺订单	业务层拒绝

集成测试走一条完整链路：注册商家 → 建店 → 建菜品 → 注册用户 → 浏览店铺菜品 → 加购 → 从购物车下单 → 商家确认/配送/完成 → 用户查询订单详情，以此验证认证、权限、业务规则、事务和数据库状态的整体正确性。

11. 设计总结

AiTakeaway 基于 Spring Boot 分层架构，把认证、商家、菜品、购物车、订单和 AI 助手做了模块化设计：概要设计上强调高内聚低耦合，详细设计上用类图、E-R 图、顺序图、状态机和流程图描述了核心业务的内部结构。整体来看，分层清晰、业务边界明确、权限控制覆盖接口层与数据归属两层、关键操作有事务保障，订单明细快照和购物车冗余字段兼顾了历史稳定与查询效率，AI 助手则作为独立能力层接入，不与数据库和核心订单逻辑强耦合。这套设计能够支撑课程项目的主要点餐场景，也为后续的推荐、支付、评价、配送等功能留出了扩展空间。